



Network Deployment and Alternative OSs

Booting over the network can be quite a challenge. However, once it is set up, it works like a charm.

Booting over the network is very useful whether you are using servers with network files storage or diskless client workstations. The idea is very simple. An operating system is installed in a directory on a network server. The client machine boots from the file server using the PXE protocol and NFS root. The server uses DHCP and an NFS server to configure the client and point it to the appropriate root directory.

The Linux Terminal Server Project (LTSP)

In the late 1990s, the LTSP was created to enable low-end PCs to function as thin clients, especially for schools. The project created server scripts and a lightweight distribution for the client machines to make it easy for deployment at schools. An even greater benefit than being able to use obsolete PCs was the simplification of support. We found it far easier to manage over 60 clients with a server in various labs and offices than maintain even a single lab with networked PCs.

In order to be able to fine tune the distribution for each server environment, the LTSP project now creates scripts that would build a client OS on the server. So, Ubuntu-based clients would be created on Ubuntu, and SUSE-based clients on SUSE. A client machine boots and starts a custom display manager called LDM, which talks to the server, finds out the sessions available and starts an X session on the server.

By default, all applications run on the server and the displays are shown on the client. The project has added capabilities for running local applications.

What makes it hard?

The scripts can be non-trivial. The same root directory may be used by many clients. So, the root should be read-only; however, some files may need to be modified specifically for the client, and some directories and files need to be writable for normal operations.

The common solution was to mount the required files and directories on tempfs. However, the recent versions are now switching to a UnionFS.

A second major change Ubuntu has made is to use a Network Block Device, by default. You create a SquashFS image of the root directory. An NBD server makes it available on the server. The client connects to it using an NBD device, though it is not clear if an NBD server offers an advantage over NFSv4.

The scripts have to keep evolving as distributions change the booting infrastructure, e.g., the replacement of *mkinitrd* by *Dracut*. The generation of a custom *initramfs* is a critical need for the diskless boot environment. LTSP is not supported on recent versions of Fedora, probably because of the switch to *systemd* from *init*.

Performance

On a 1 Gbps network, a diskless workstation was logged in and ready to use in just under a minute. The additional time needed on a 100 Mbps network was less than 15 seconds. The major difference in network speed is evident in how snappy a user's environment is after signing in. This is because while the X server is running on the client, all applications, including the window manager, are running on the server. There are a lot of network traffic and resultant latency issues.

Modern desktop environments with desktop effects like Unity or the Gnome3 shell can be painfully unresponsive on the diskless environment. The fallback mode of Gnome or KDE with desktop effects disabled, runs pretty well on diskless environments.

Media-rich applications may be impacted by the network delays. Hence, LTSP makes it easy to use local applications. You install local applications on the server in the change-root environment. For instance, you can install Firefox and Flash Player for an excellent browsing experience.

The window manager and panel are running on the server. So, running a local application requires a special script – *ltsp-localapps*. It makes very clever use of the *xprop* command of the X server to run the command on the local system.

In case you want to experience the feel of a diskless session without setting up the diskless environment, you may do what I routinely do. You will need two machines—the